

REST に全部賭けたあなたのための Remote Procedure Call 入門

RPC とは

ネットワーク越しにある関数を、ローカル関数と同じように呼び出せる仕組み

```
# この実装をネットワーク越しでも呼び出せるようにしたい  
user = get_user(123)
```

RPC の歴史

初期の RPC

- TCP 上の独自プロトコル (ONC RPC など)
- OS や言語専用の RPC (Java RMI など)

課題: 多様な環境に対応しづらい

現在: HTTP ベースの RPC が主流

- gRPC、ConnectRPC など

HTTP ベース RPC のメリット

- 既存の Web インフラが使える
 - ロードバランサ、トレース、認証、監視
- ファイアウォールやブラウザとの親和性
- REST API に慣れた開発者が学習しやすい

REST vs RPC

観点	REST	RPC
基本概念	リソースの操作	手続きの実行
URI	<code>/users/123</code>	<code>/example.v1.UserService/GetUser</code>
操作の表現	HTTPメソッド + リソースURI	メソッド名（関数）

REST vs RPC: 具体例

REST

```
GET /users/123
```

RPC

```
GetUser(123)
```

REST は「何に対して何をしたいか」

RPC は「どの機能を実行したいか」

データ形式と契約の違い

項目	REST	RPC
データ形式	JSON 等（疎な型定義）	Protocol Buffers 等（厳密）
契約共有	ドキュメント依存になりがち	IDL による明確な契約が前提
自動生成	追加ツールが必要	IDL からコード生成

※ Interface Definition Language (IDL): 言語に依存しないインターフェイス定義

Protocol Buffers (Protobuf)

Google が開発したシリアライズフォーマット

- データ構造と RPC メソッドを定義
- バイナリ形式で高速・軽量
- `.proto` ファイルから多言語コード生成
- gRPC / ConnectRPC は `.proto` でインターフェイスを定義する

Protocol Buffers の例

```
message GetUserRequest {  
    int64 id = 1;  
}  
  
message User {  
    int64 id = 1;  
    string name = 2;  
}  
  
service UserService {  
    rpc GetUser(GetUserRequest) returns (User);  
}
```

gRPC

Google が開発した高速・型安全な RPC フレームワーク

技術基盤

- HTTP/2: 多重化、ヘッダ圧縮、双方向ストリーミング
- Protocol Buffers: バイナリ形式で軽量・高速

`.proto` から主要言語（Go, Python, Java, TypeScript 等）のクライアント/サーバーコードを自動生成できるのが魅力

gRPC の強み

- 高速・低レイテンシ
- 双方向ストリーミング
- 多言語対応
- 契約の一元管理
- マイクロサービスとの相性

gRPC の課題

- ブラウザから直接呼び出しにくい
- gRPC-Web + プロキシが必要
- バイナリ形式のためデバッグ性が低い（通信内容が読めない）
- HTTP/1.1 環境との互換性が低い

→ **ConnectRPC が登場**

ConnectRPC

gRPC の強みはそのままに、Web で扱いやすく進化

コアコンセプト

1. シンプルな HTTP RPC
2. どのクライアントからでもアクセス可能
3. 明確な仕様と開発者体験の向上

ConnectRPC の特徴

- ブラウザ対応: プロキシ不要で Web から直接呼び出し
- HTTP/1.1 対応: レガシー環境にも適用可能
- デバッグ性向上: Protobuf に加えて JSON にも対応、curl で検証可能
- gRPC 互換: 既存の `.proto` がそのまま使える
- 複数プロトコル対応: gRPC / gRPC-Web / Connect を切り替え可能

実装例: 前提

必要なもの

- Python
- uv
- Buf CLI

```
$ uv add connect-python protobuf uvicorn
```

1. Protobuf 定義

proto/hello.proto

```
syntax = "proto3";

package example.v1;

message HelloRequest {
    string name = 1;
}

message HelloResponse {
    string message = 1;
}

service HelloService {
    rpc SayHello(HelloRequest) returns (HelloResponse);
}
```


2. Buf 設定

buf.gen.yaml

```
version: v2
plugins:
  - remote: buf.build/protocolbuffers/python
    out: .
  - remote: buf.build/protocolbuffers/pyi
    out: .
  - remote: buf.build/connectrpc/python
    out: .
```

buf.yaml

```
version: v2
modules:
  - path: proto
```

3. コード生成

```
$ buf generate
```

生成されるファイル:

- `hello_pb2.py` : メッセージクラス
- `hello_pb2.pyi` : 型ヒント
- `hello_connect.py` : サービス/クライアントクラス

4. サーバー実装

```
from connectrpc.request import RequestContext
from hello_pb2 import HelloRequest, HelloResponse
from hello_connect import HelloService, HelloServiceASGIApplication

class MyHelloService(HelloService):
    async def say_hello(
        self, request: HelloRequest, ctx: RequestContext
    ) -> HelloResponse:
        return HelloResponse(message=f"Hello, {request.name}!")

app = HelloServiceASGIApplication(MyHelloService())
```

5. サーバー起動

```
$ uvicorn server:app --host 0.0.0.0 --port 8000
```

6. 動作確認 (curl)

エンドポイント: `POST /<package>.<Service>/<Method>`

```
$ curl -X POST http://localhost:8000/example.v1.HelloService/SayHello \  
  -H "Content-Type: application/json" \  
  -d '{"name": "Alice"}'
```

```
{  
  "message": "Hello, Alice!"  
}
```

7. クライアント実装

```
import asyncio
from hello_pb2 import HelloRequest
from hello_connect import HelloServiceClient

HELLO_SERVICE_URL = "http://hello-service:8000"

async def call_hello():
    async with HelloServiceClient(HELLO_SERVICE_URL) as client:
        req = HelloRequest(name="another microservice")
        res = await client.say_hello(req)
        print(res.message)

asyncio.run(call_hello())
```

RPC の利点（再確認）

```
# REST の場合
response = await http_client.get(
    "http://service/users/123",
    headers={"Content-Type": "application/json"}
)

# RPC の場合
user = await client.get_user(GetUserRequest(id=123))
```

URI やヘッダを意識せず、関数呼び出し感覚で利用可能

まとめ

- **RPC:** ネットワーク越しの関数呼び出しを抽象化
- **REST vs RPC:** リソース指向 vs 手続き指向
- **Protocol Buffers:** `.proto` がスキーマの正となる、多言語コード生成
- **gRPC:** 高速・型安全だが Web 対応が課題
- **ConnectRPC:** gRPC の強みを保ちつつ Web フレンドリー

参考リンク

- [ConnectRPC](#)
- [gRPC](#)
- [Protocol Buffers](#)
- [Buf](#)